

Comparison of Adaptive Traffic Light Management Systems Using YOLOv8n and YOLO26n

Katrina Kayla Kartino
School Of Computer Science
Bina Nusantara University
Sentul, Indonesia
katrina.kartino@binus.ac.id

Davina Verena Ancilla Paendong
School Of Computer Science
Bina Nusantara University
Sentul, Indonesia
davina.paendong@binus.ac.id

Annona Benedicta
School Of Computer Science
Bina Nusantara University
Sentul, Indonesia
annona.benedicta@binus.ac.id

Jeremy Christopher Amadeus Widyanto
School Of Computer Science
Bina Nusantara University
Sentul, Indonesia
jeremy.widyanto@binus.ac.id

Matthew Anthony Subagyo
School Of Computer Science
Bina Nusantara University
Sentul, Indonesia
matthew.subagyo@binus.ac.id

Abstract—To be continue...

Keywords—To be continue...

I. INTRODUCTION

Traffic congestion has become a critical global challenge faced by major metropolitan areas worldwide. Rapid urbanization further exacerbates congestion levels and drives a surge in public mobility activities. According to data from the TomTom Traffic Index, global megacities continue to experience a significant rise in traffic congestion. [1] Bandung City, serving as the governmental and economic hub of West Java, ranks as the second most congested city in Asia and fifth globally. [2] This highlights the pressing urgency to reform the transportation sector.

On the other hand, road infrastructure has long been a priority for the Indonesian government due to its essential role as a national connector. The Indonesian government, through the Ministry of Finance, has collaborated with enterprise entities to construct roads that adhere to established standards. [3] Although standardized roads have been successfully realized, high population density and mobility in Indonesia still present a persistent potential for traffic congestion. Consequently, an absolute solution is required to address this ongoing issue.

In practice, the Indonesian government has implemented several traffic management schemes across various major cities, including the digitization of traffic signal systems via Area Traffic Control System (ATCS) technology. [4] The ATCS integrates traffic lights across major intersections, allowing signal durations to be monitored and managed centrally to alleviate vehicle queueing. Additionally, the Jakarta provincial government introduced traffic restriction measures by enhancing public mass transit facilities and implementing a one-way traffic system. [5] However, given the continuously increasing volume of motor vehicles, these conventional policies are considered to have produced a limited impact.

To overcome the limitations of these conventional solutions, a transformation toward an Intelligent Transportation System (ITS) leveraging Artificial Intelligence (AI)—specifically Computer Vision—is essential. By employing Computer Vision, existing Closed-Circuit Television (CCTV) cameras deployed at intersections can be repurposed to detect, count, and classify vehicle volumes in real time. This visual detection data can subsequently be

integrated into the ATCS to automatically adjust traffic light durations (adaptive traffic control) based on actual density levels in each lane. [6]

In implementing a Computer Vision system, selecting an appropriate object detection model represents a crucial factor, as different models possess distinct characteristics in terms of accuracy, inference speed, and computational requirements. This study utilizes the You Only Look Once (YOLO) framework, which is widely recognized for its high performance in real-time object detection. [7] Over time, YOLO has undergone numerous iterative updates, each offering performance enhancements with distinct operational trade-offs. Therefore, this research conducts a comparative analysis to determine the most suitable model for CCTV-based vehicle detection systems. Specifically, this study aims to evaluate and compare multiple YOLO versions based on detection accuracy and inference latency to identify the optimal model for supporting adaptive traffic light control systems.

Furthermore, the development of this YOLO-based Intelligent Transportation System aligns with the United Nations Sustainable Development Goals (SDGs). Deploying AI-driven innovations directly supports SDG 9 (Industry, Innovation, and Infrastructure) by fostering resilient transportation infrastructure and advancing cutting-edge technological innovation. Additionally, the system contributes directly to SDG 11 (Sustainable Cities and Communities) by reducing waiting times caused by traffic jams, mitigating tailpipe emissions from idling vehicles, and establishing a safer, smarter, and more inclusive urban mobility framework. [8]

II. BACKGROUND AND LITERATURE REVIEW

A. Computer Vision

Computer vision is a branch of artificial intelligence which allows computers to interpret and understand visual information from images and video footage. By using a combination of various traditional image processing techniques and also deep learning algorithms, computer vision systems have become able to detect, classify, track, and also analyze objects automatically in real time. [12]

In Intelligent Transportation Systems (ITS), computer vision has been used for vehicle detection, traffic density estimation, vehicle counting, traffic monitoring, and queue length

analysis using surveillance cameras. When compared to using traditional sensors, camera-based systems offer a plethora of benefits, such as a denser flow of information and also relatively lower infrastructure costs.

B. Intelligent Transportation System (ITS)

Intelligent Transportation Systems (ITS) are systems that implement advanced communication technologies, sensors, and intelligent algorithms to increase efficiency, safety, and also the sustainability of modern traffic systems. One of the primary goals of ITS is to actively monitor traffic conditions in real time and to help make better and more informed traffic management decisions. [13]

Computer Vision has become a vital component of modern ITS systems as cameras provide a more meaningful stream of data compared to using traditional sensors. Object-based detection systems based on deep learning technology allow for the active estimation of traffic density, vehicle counting, and also queue lengths. All those factors are crucial for the proper management of traffic and also adaptive traffic light systems. [14]

C. YOLO (You Only Look Once)

YOLO (You Only Look Once) is a family of object detection algorithms owned and developed by the American company Ultralytics. At its core, YOLO is a one stage object detection algorithm that does object localization and classification simultaneously in one forward pass. When compared to two stage detection like Faster R-CNN, YOLO has a much faster inference time whilst still maintaining a competitive detection accuracy. This speed makes YOLO much more compatible for use in real time systems. [15]

Since its introduction, the YOLO family of algorithms have evolved through several generations, each bringing its own array of performance and efficiency upgrades. Through its balance of speed and accuracy, YOLO has become one of, if not, the most used object detection algorithms for real time applications like self-driving, surveillance, robotics, and traffic monitoring systems. [16][17]

D. YOLOv8n

YOLOv8 is the eighth generation of YOLO object detection algorithms developed by Ultralytics. Released in early 2023, this model introduced an anchor-free detection mechanism and a decoupled detection head, both those improvements allowed for increased localization accuracy whilst reducing computational complexity. [18]

YOLOv8n (Nano) is the smallest version in the YOLOv8 family. This version of the YOLO model was designed for use with devices with limited computational resources. Despite its lightweight design, YOLOv8n still provides decent detection accuracy and real time inference. [19][20]

E. YOLO26n

YOLO26 is the latest generation of the YOLO object detection algorithm developed by Ultralytics. Released in early 2026, YOLO26 deviates from its predecessors in the fact that it removed its dependency on Non-Maximum Suppression (NMS) during inference. This change allowed for lower latency and a simpler detection pipeline. YOLO26 also introduced several architectural upgrades such as Progressive Loss (ProgLoss) and Small-Target-Aware Label Assignment (STAL), which allowed for increased stability during training and improved small object detection. [21]

YOLO26n is the lighter version in the YOLO26 family which is better optimized for faster inference whilst maintaining detection accuracy on lightweight and resource limited devices. [21][22]

F. Research Gap

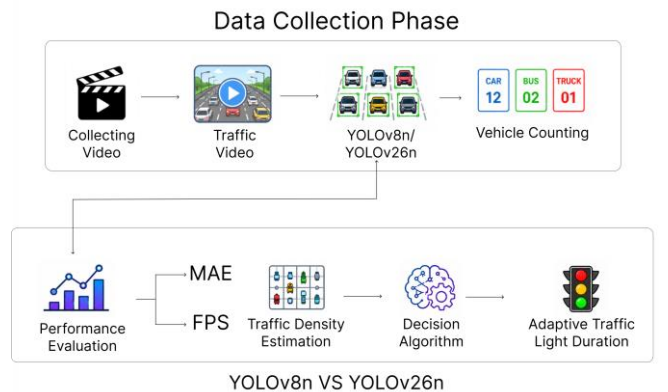
While deep learning-based object detection models, in this case the YOLO family of models, have been widely used, publicized, and researched for its real-world usage, most studies are primarily focused on older versions of YOLO, primarily YOLOv5, YOLOv7, and YOLOv8. On the other hand, the latest version of the YOLO family, YOLO26, has only just been released for a few months and has not been extensively researched in traffic related applications. Although the official publication reports improvements in both efficiency and accuracy, there is very limited evidence of its improvements in real world scenarios and systems. [23] Furthermore, only a handful of studies directly compare the lightweight n (Nano) versions of the YOLO model. Not only that, but the performance metrics in traffic surveillance applications also using YOLOv8n and YOLO26n are not explored very thoroughly.

Therefore, we aim to address this gap by doing a comparative evaluation between the performance of YOLOv8n and YOLO26n for traffic queue detection. The comparison will analyze both models based on their detection performance and their computational efficiency to determine their suitability for real-time Intelligent Transportation System applications.

III. RESEARCH METHODOLOGY

In most past research, traffic surveillance applications have primarily focused on older architectures such as YOLOv5, YOLOv7, and YOLOv8, leaving newer lightweight models like YOLO26n largely underexplored [23]. To address this gap, this study proposes a comparative evaluation between YOLOv8n and YOLO26n for traffic queue detection. By analyzing both models in terms of detection accuracy and computational efficiency, this research aims to determine the most optimal lightweight architecture for real-time Intelligent Transportation Systems.

A. Research Framework



B. Dataset

The dataset used in this study consists of traffic video footage obtained from YouTube traffic recordings and recorded CCTV footage. The videos represent real-world traffic conditions with different levels of vehicle density, allowing the proposed system to evaluate vehicle detection

under varying traffic conditions. Four traffic videos were used as input, representing heavy, medium, and light traffic conditions.

The system focuses on four vehicle categories that are relevant to traffic monitoring and queue estimation: car, motorcycle, bus, and truck. These vehicle classes correspond to the vehicle categories available in the pretrained YOLO models used in this study. During inference, only these four classes are considered, while other object categories are ignored. A confidence threshold of 0.6 is applied to filter out detections with low confidence.

Unlike conventional object detection studies that train a model using an annotated image dataset, this study uses the pretrained YOLO models directly for inference without additional training or fine-tuning. Therefore, the videos are not divided into training, validation, and testing subsets. Instead, the same video inputs and detection pipeline are provided to both YOLOv8n and YOLO26n to ensure a fair comparison between the two models.

Each video is processed frame by frame using an input resolution of 640×640 pixels. The detected vehicles are subsequently tracked using ByteTrack to maintain their identities across consecutive frames. The center point of each detected vehicle is then used to determine whether the vehicle is located within a predefined queue region. Vehicles whose center points fall within the designated region are counted as part of the traffic queue.

The resulting vehicle count represents the traffic condition at each monitored intersection or traffic direction. This information serves as the primary input for the adaptive traffic light system, where traffic density can be used to determine an appropriate green-light duration. Through this approach, the dataset is not only used to evaluate vehicle detection performance but also to simulate the real-time traffic monitoring process required for an adaptive traffic signal control system.

C. Model Architecture

This study compares two lightweight object detection models, YOLOv8n and YOLO26n, for real-time vehicle detection in an adaptive traffic light system. Both models belong to the Nano (n) variant of their respective YOLO generations and are designed to provide a balance between detection performance and computational efficiency. The models are evaluated under the same detection and tracking pipeline to ensure a fair comparison.

1) YOLOv8n

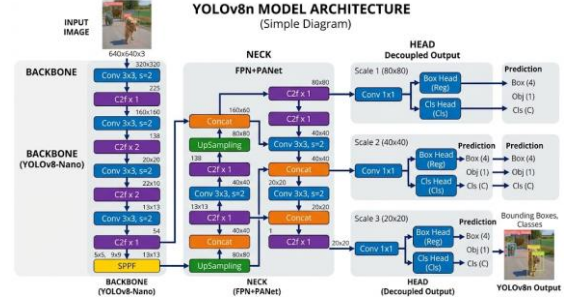
Architecture

YOLOv8n follows the general three-stage YOLO architecture consisting of a backbone, neck, and detection head. **The backbone extracts hierarchical visual features, while the neck combines feature representations across different spatial scales, while a decoupled head predicts the location and class of objects.**

C2f (Cross-Stage Partial Bottleneck with two convolutions) modules, concatenate immediate bottleneck outputs, improving gradient flow while maintaining a lightweight architecture.

For predictions, an anchor-free, decoupled detection head is used to process classification and bounding-box

regression in separate branches, using Distribution Focal Loss (DFL) to optimize regression.

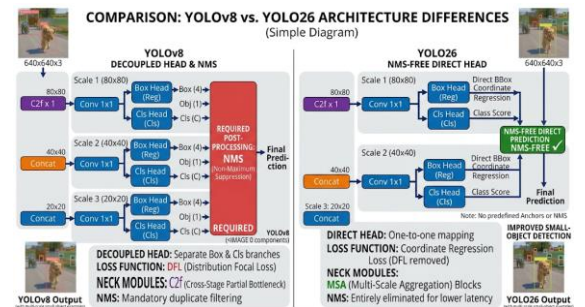


Why YOLOv8n was chosen

YOLOv8n was selected as the main model in this study because YOLOv8n remains a widely adopted and well-established YOLO architecture with competitive performance in real-time object detection. Its Nano variant provides a lightweight model that is suitable for vehicle detection while maintaining a balance between detection accuracy and computational efficiency. Therefore, YOLOv8n serves as a representative baseline for evaluating whether the newer YOLO26n architecture can provide improvements in real-time traffic monitoring.

• YOLO26n

The primary difference between YOLOv8n and YOLO26n lies in the detection head and post-processing pipeline. Where v8n uses a decoupled-head and bounding-box regression, relying on DFL and requiring NMS post-processing to filter out duplicate bounding boxes. Meanwhile, YOLO26 replaces the head with one that outputs final bounding boxes directly from grid points, eliminating both DFL and the NMS bottleneck for lower edge latency. YOLO26 also simplifies the backbone and neck by swapping out YOLOv8's C2f modules for lightweight Multi-Scale Aggregation (MSA) blocks.



The table below summarizes the main architectural differences between YOLOv8n and YOLO26n.

Components	YOLOv8n	YOLO26n
Backbone block	C2f	C3k2
Spatial Feature Processing	SPPF	SPPF + Shortcut
Attention	-	C2PSA

Detection Head	Anchor-free, decoupled	Anchor-free, NMS-free, end-to-end
DFL	Used	Removed
Parameters	3.15 M	2.41 M
FLOPs	8.7 B	5.4 B

Why YOLO26n Was Compared with YOLOv8n

YOLO26n was selected as the comparison model because it represents a newer generation of lightweight YOLO architecture while maintaining the same Nano model scale. This makes the comparison more meaningful than comparing models with substantially different sizes and computational requirements.

YOLOv8n serves as an established lightweight baseline, whereas YOLO26n introduces architectural changes specifically aimed at improving the accuracy–latency trade-off and deployment efficiency. The comparison therefore investigates whether these architectural improvements translate into measurable advantages when applied to real-time traffic videos.

Both models are evaluated using the same traffic videos, input resolution, vehicle classes, confidence threshold, tracking method, and computational environment. The comparison focuses primarily on detection accuracy and inference speed, since both factors directly influence the ability of the proposed system to monitor traffic conditions and respond to changing vehicle queues in real time.

The comparison can therefore be represented as:

YOLOv8n (Established Lightweight Baseline)

↓

Same Dataset + Same Pipeline + Same Hardware

↓

YOLO26n (Newer Lightweight Architecture)

↓

Comparison of Detection Accuracy and Inference Speed

↓

Determine Suitability for Adaptive Traffic Light Systems

YOLO26n was selected as a newer lightweight architecture to be compared with the established YOLOv8n baseline. Both models were evaluated under the same experimental conditions, with detection accuracy and inference speed used as the primary comparison metrics.



D. Vehicle Detection and Counting

The vehicle detection and counting stage is responsible for estimating the number of cars approaching each direction of the intersection using prerecorded traffic videos. Four traffic video streams representing the north, south, east, and west approaches are processed independently to obtain the traffic condition of each lane.

Each video stream is processed frame by frame and provided as input to the selected object detection model, either YOLOv8n or YOLO26n. For every frame, the model

performs inference to detect vehicles and generates the corresponding bounding boxes, confidence scores, and class labels. Since this study focuses exclusively on passenger cars, only objects classified as cars are retained, while all other detected objects are discarded.

To estimate the queue length, only detected cars located within a predefined Region of Interest (ROI) near the stop line are counted. The ROI represents the waiting area where vehicles queue before entering the intersection. The resulting vehicle count for each approach is then used to estimate the current traffic density and serves as the input to the adaptive traffic light decision making algorithm.

E. Decision-Making Algorithm for Adaptive Traffic Light Control

The proposed decision-making algorithm determines the green-light duration based on the number of detected cars at each traffic approach. The vehicle count obtained from the detection stage is used to estimate the traffic density and calculate the duration of the green signal.

The green-light duration is determined using a linear interpolation between the predefined minimum and maximum green durations, as expressed in:

$$G = G_{min} + N \cdot \frac{G_{max} - G_{min}}{N_{max}}$$

Where G denotes the green light duration, G_{min} and G_{max} represent the minimum and maximum green durations, respectively, N is the detected number of cars, and N_{max} is the predefined maximum vehicle count.

After the green light duration is calculated, the corresponding traffic approach is allowed to proceed for the assigned duration before the signal transitions to the next phase. This process is repeated continuously as new vehicle counts are obtained from the detection stage.

In addition, when no vehicles are detected at any traffic approach, all traffic signals remain in the red state. Once a vehicle is detected in one of the approaches, the corresponding traffic signal is immediately activated, allowing traffic to resume without waiting for a complete signal cycle.

F. Performance Evaluation

To assess the proposed adaptive traffic management system, YOLOv8n and YOLO26n were evaluated across two key performance indicators: Mean Absolute Error (MAE) and Frames Per Second (FPS). Together, these metrics capture both the accuracy of vehicle counting and the computational speed necessary for real-time monitoring.

Vehicle counting accuracy was measured using Mean Absolute Error (MAE), which calculates the average difference between the model's detected vehicle count and the actual ground-truth value. A smaller MAE reflects a prediction that closely aligns with real road conditions, signaling higher reliability. MAE is defined as follows:

where y_i represents the ground-truth vehicle count for frame i , $\hat{y}_i$ denotes the count predicted by the model, and n is the total number of evaluated frames.

Computational performance was evaluated using Frames Per Second (FPS), which tracks how many video frames the model can process each second during inference. Higher FPS values indicate faster processing capacity—a critical

requirement for real-time adaptive signals that rely on immediate traffic data to adjust timing cycles.

Ultimately, the combined results from MAE and FPS offer a complete picture of how YOLOv8n and YOLO26n balance accuracy against processing speed. The model that delivers a lower MAE while maintaining a sufficiently high FPS is deemed best suited for real-world adaptive traffic light control.

IV. RESULT AND DISCUSSION

To be continue..

V. CONCLUSION

To be continue...

REFERENCES

- [1] TomTom, "TomTom Traffic Index – City Ranking," TomTom Traffic Index. [Online]. Available: <https://www.tomtom.com/traffic-index/ranking/>. [Accessed: Aug. 4, 2026].
- [2] TomTom, "TomTom Traffic Index – Indonesia," TomTom Traffic Index. [Online]. Available: <https://www.tomtom.com/traffic-index/country/indonesia/>. [Accessed: Aug. 4, 2026].
- [3] Kementerian Keuangan Republik Indonesia, "Mengenal Jalan sebagai Infrastruktur Penyangga Logistik dan Konektivitas Nasional," KPBU Kementerian Keuangan. [Online]. Available: <https://kpbu.kemenkeu.go.id/read/1182-1534/umum/kajian-opini-publik/mengenal-jalan-sebagai-infrastruktur-penyangga-logistik-dan-konektivitas-nasional>. [Accessed: Aug. 4, 2026].
- [4] Area Traffic Control System (ATCS) Kabupaten Cianjur, "Area Traffic Control System (ATCS) Kabupaten Cianjur." [Online]. Available: <https://atcs.cianjurkab.go.id/>. [Accessed: Aug. 4, 2026].
- [5] Jakarta Smart City, "Jurus-Jurus Jitu Jakarta Hadapi Kemacetan," Jakarta Smart City. [Online]. Available: <https://smartcity.jakarta.go.id/blog/jurus-jurus-jitu-jakarta-hadapi-kemacetan/>. [Accessed: Aug. 4, 2026].
- [6] Intel Corporation, "Apa Itu Computer Vision?" Intel Indonesia. [Online]. Available: <https://www.intel.co.id/content/www/id/id/learn/what-is-computer-vision.html>. [Accessed: Aug. 4, 2026].
- [7] Ultralytics, "Ultralytics Documentation," Ultralytics Docs. [Online]. Available: <https://docs.ultralytics.com/>. [Accessed: Aug. 4, 2026].
- [8] United Nations, "The 17 Goals | Sustainable Development," United Nations Sustainable Development Goals. [Online]. Available: <https://sdgs.un.org/goals>. [Accessed: Aug. 4, 2026].
- [9] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Pearson, 2021.
- [10] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. MIT Press, 2016.
- [11] Y. LeCun, Y. Bengio, and G. Hinton, "Deep Learning," Nature, vol. 521, no. 7553, pp. 436–444, 2015.
- [12] R. Szeliski, Computer Vision: Algorithms and Applications, 2nd ed. Springer, 2022.
- [13] L. Vlacic, M. Parent, and F. Harashima, Intelligent Vehicle Technologies. Elsevier, 2001.
- [14] Ü. Özgüner, K. Redmill, and A. Broggi, "Intelligent vehicles," in Springer Handbook of Robotics. Springer, 2016.
- [15] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2016, pp. 779–788.
- [16] J. Redmon and A. Farhadi, "YOLO9000: Better, Faster, Stronger," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2017, pp. 6517–6525.
- [17] J. Redmon and A. Farhadi, "YOLOv3: An Incremental Improvement," arXiv:1804.02767, 2018.
- [18] G. Jocher et al., "Ultralytics YOLOv8," 2023.
- [19] G. Jocher, A. Chaurasia, and J. Qiu, Ultralytics YOLO Documentation, 2023.
- [20] G. Jocher et al., "Ultralytics YOLO26: Unified Real-Time End-to-End Vision Models," arXiv:2606.03748, 2026.
- [21] G. Jocher and J. Qiu, Ultralytics YOLO26 Documentation, 2026.